EXAMEN PRÁCTICO · SEGUNDO PARCIAL

# Analizador de red de colaboradores

Diseño, algoritmos, pruebas y resultados de una herramienta de análisis de grafos en Java 17.

| 14 | 16 | 2 | 8/8 |
|---|---|---|---|
| colaboradores | relaciones dirigidas | componentes débiles | pruebas aprobadas |

## Resumen ejecutivo

Se desarrolló una aplicación de consola que carga relaciones desde CSV, mantiene el grafo dirigido y su proyección no dirigida, y calcula las métricas solicitadas. La solución incluye componentes conexas débiles, ciclos, grados de entrada y salida, un camino simple más largo entre dos vértices y puntos de articulación. El programa imprime una salida tabulada y la exporta a resultados.txt.

> Resultado de verificación
> La solución compila para Java 17 y las ocho pruebas JUnit 5 ejecutadas finalizaron correctamente.
> El archivo de ejemplo contiene 14 colaboradores distribuidos en dos componentes.

Entregables incluidos: código Java documentado, pruebas JUnit 5, archivo de entrada red.csv, resultados.txt y este reporte.

Fecha de elaboración: 23 de octubre de 2026

# 1. Alcance y modelado

El problema representa colaboradores como vértices y relaciones de colaboración o dependencia como aristas. Para atender las consultas en ambos sentidos, cada arista se registra en dos estructuras relacionadas.

| Requisito | Implementación |
|---|---|
| RF1 · Carga | CSV UTF-8 por su sencillez de inspección y edición. Parser propio en Java que reconoce comillas, comas dentro de campos, comillas escapadas, comentarios y filas de nodos/aristas. |
| RF2 · Recorridos y ciclos | DFS y BFS para componentes conexas débiles; DFS de tres colores para ciclos dirigidos y BFS con seguimiento del padre para ciclos no dirigidos. |
| RF3 · Grados | Conteo de grados de entrada y salida mediante listas de adyacencia, incluyendo vértices aislados y auto-lazos. |
| RF4 · Ruta simple más larga | Backtracking dirigido entre origen y destino, con una cota superior basada en los vértices aún no visitados. |
| RF5 · Articulaciones | DFS de Tarjan sobre el grafo no dirigido derivado, con tiempos de descubrimiento y valores low-link. |
| RF6 · Salida | Tablas separadas por tabuladores en consola y exportación del mismo contenido a resultados.txt. |

## Decisión de representación

Se conserva una lista de adyacencia dirigida para los grados, ciclos dirigidos y rutas. Al insertar A → B, también se agregan A ↔ B a una segunda lista no dirigida. Esta proyección define componentes débiles, ciclos no dirigidos y articulaciones. Las aristas tienen peso unitario; las relaciones duplicadas se colapsan y los auto-lazos se conservan.

## CSV y estructuras

El archivo distingue filas nodo,id y arista,origen,destino. Así se pueden expresar colaboradores aislados sin crear relaciones ficticias. HashMap indexa la adyacencia; HashSet evita aristas y visitas duplicadas; ArrayDeque implementa colas y pilas de recorrido. ArrayDeque evita la sincronización innecesaria de Stack. LinkedList crea un nodo enlazado por elemento, con más objetos y peor localidad para estos recorridos.

Las componentes se interpretan como conexas débiles: dos colaboradores pertenecen al mismo grupo si están conectados al ignorar la dirección de las relaciones.

# 2. Algoritmos y eficiencia

Se recorren listas de adyacencia para que los algoritmos dependan del número de vértices y aristas, en lugar de inspeccionar todas las parejas posibles.

| Análisis | Método y criterio | Complejidad |
|---|---|---|
| Componentes | DFS iterativo y BFS sobre la proyección no dirigida; cada vértice se marca al encolarse o apilarse. | O(V + E) base |
| Ciclo dirigido | DFS iterativo de tres colores: gris identifica una arista de retroceso; negro indica que el vértice ya terminó. | O(V + E) base |
| Ciclo no dirigido | BFS con padre por vértice; una conexión con un visitado que no sea el padre revela un ciclo. El auto-lazo cuenta como ciclo. | O(V + E) base |
| Grados | La salida es el tamaño de cada lista dirigida; la entrada se acumula al visitar cada arista. | O(V + E) base |
| Articulaciones | Tarjan compara low-link de cada hijo con el tiempo de descubrimiento; se aplica una regla especial para la raíz. | O(V + E) base |
| Camino simple más largo | Backtracking dirigido sin repetir vértices; poda si la ruta actual más el máximo aporte restante no mejora el resultado. | O(V! log V) implementación |

## Cota de poda y límite práctico

Si la ruta actual tiene k aristas y quedan r vértices sin visitar, como máximo se pueden agregar r aristas a un camino simple. Cuando k + r no supera la mejor longitud conocida, la rama se descarta. La poda reduce búsquedas en muchos grafos, pero no elimina el peor caso. Encontrar un camino simple más largo entre dos vértices es NP-difícil en general. El número de permutaciones posibles es factorial; el ordenamiento de vecinos añade un factor logarítmico en la implementación reproducible.

Los recorridos base de listas de adyacencia son O(V + E). Para repetir el mismo orden entre ejecuciones, esta implementación ordena vecinos durante los recorridos y vértices al construir algunas salidas. Ese ordenamiento añade hasta O(V log V + E log V); la ruta más larga sigue teniendo un costo factorial en el peor caso y usa O(V) espacio de recorrido, sin contar el grafo.

# 3. Pruebas y casos borde

La suite JUnit 5 cubre los escenarios obligatorios del enunciado y agrega comprobaciones para grados, búsqueda de rutas y lectura CSV. Las pruebas se ejecutaron con JUnit Platform Console Standalone 1.11.4, que contiene JUnit Jupiter 5.11.4, la versión declarada en el proyecto.

| Caso | Validación principal | Resultado |
|---|---|---|
| Grafo con ciclo | Ciclo dirigido y no dirigido; un componente. | Correcto |
| Grafo en árbol | Sin ciclos; B es punto de articulación. | Correcto |
| Dos componentes | DFS y BFS devuelven los mismos dos grupos. | Correcto |
| Un nodo | Un componente, grados cero y ruta de longitud cero. | Correcto |
| Red vacía | Sin componentes, ciclos, ruta ni articulaciones. | Correcto |
| Auto-lazo | Ambos ciclos presentes; entrada y salida iguales a uno. | Correcto |
| Ruta y grados | Ruta A → B → C → D; grados y ausencia de ruta Z → A. | Correcto |
| Lectura CSV | Nodo aislado y nombre con coma entrecomillado. | Correcto |

| | | |
|---|---|---|
| 8 pruebas aprobadas | 0 fallidas | 0 omitidas |

## Compilación

Las clases de producción y prueba compilaron con --release 17 en un JDK 25. El proyecto también incluye pom.xml para ejecutar la misma suite mediante mvn test. No se usan bibliotecas externas de grafos.

# 4. Resultados de ejemplo

La red red.csv registra 14 colaboradores y 16 aristas dirigidas. El análisis reporta dos componentes conexas débiles: una de 11 integrantes y otra de 3. Se detectan ciclos en ambos grafos: las direcciones forman ciclos y la proyección no dirigida contiene conexiones cíclicas.

Camino simple más largo entre Ada y Kira: Ada → Beto → Dario → Eva → Gina → Hugo → Iris → Juan → Kira, con 8 aristas. Puntos de articulación: Eva y Mateo.

## Grados por colaborador

| Colaborador | Entrada | Salida |
|---|---|---|
| Ada | 0 | 2 |
| Beto | 1 | 1 |
| Cora | 2 | 1 |
| Dario | 2 | 1 |
| Eva | 2 | 2 |
| Fabio | 1 | 1 |
| Gina | 2 | 1 |
| Hugo | 1 | 1 |
| Iris | 1 | 2 |
| Juan | 1 | 1 |
| Kira | 1 | 1 |
| Lia | 0 | 1 |
| Mateo | 1 | 1 |
| Nico | 1 | 0 |

## Ejecución reproducible

mvn test ejecuta la suite. Después de compilar, este comando vuelve a generar el archivo de ejemplo:

```
java -cp target/classes mx.edu.estructuras.red.Main red.csv resultados.txt Ada Kira
```

La salida tabulada incluye componentes, ciclos, grados, ruta, articulaciones y complejidad; se muestra en consola y se guarda como resultados.txt. El parser valida las columnas y señala el archivo y la línea si el CSV tiene un registro inválido.

## Conclusión

La solución satisface los requerimientos funcionales con estructuras de adyacencia acordes al modelo, algoritmos estándar y casos borde verificables. El límite temporal dominante es la búsqueda exacta del camino más largo por su naturaleza combinatoria.